LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 17

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada praktikum sebelumnya, kode API ditulis dalam satu berkas sehingga sulit dirawat. Acara 17 membangun API CRUD (Create, Read, Update, Delete) dengan PHP Native memakai pola MVC (Model-View-Controller), yaitu memisahkan kode menjadi Model (urusan database), Controller (logika endpoint), serta Router dan Front Controller (pintu masuk dan pengarah request). PHP Native berarti PHP murni tanpa framework seperti Laravel atau CodeIgniter.

2. Tujuan

1. Membuat struktur proyek MVC dan database db_workshop.

2. Membuat Model, Controller, Autoloader, Router, dan Front Controller.

3. Menguji operasi POST, GET, PUT, dan DELETE.

4. Mengerjakan tugas mandiri: resource Buku, validasi NIM duplikat, dokumentasi endpoint, dan diagram alur.

3. Dasar Teori

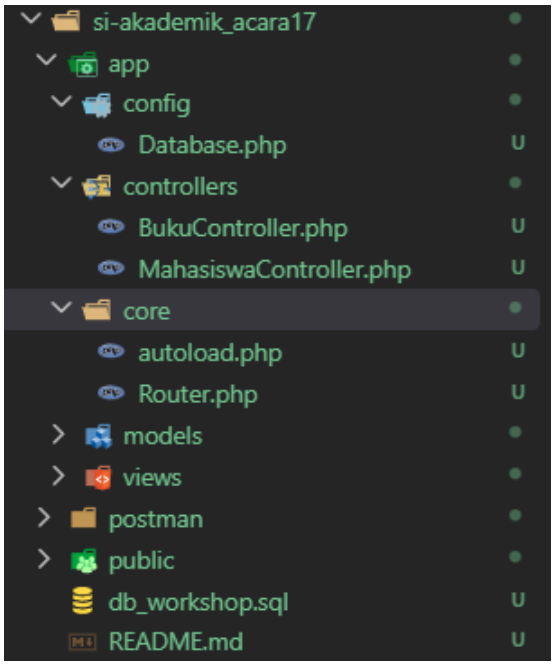
API CRUD memetakan empat operasi data ke HTTP method: Create (POST), Read (GET), Update (PUT/PATCH), dan Delete (DELETE). Pada MVC, Model mengurus data dan query, Controller menerima request lalu menentukan response, dan Router menghubungkan URL ke Controller. Autoloader (spl_autoload_register) memuat class otomatis, Front Controller (public/index.php) menjadi satu-satunya pintu masuk, dan PDO dengan prepared statement mencegah SQL Injection. Data dikirim dan diterima dalam format JSON (json_encode dan php://input).

HASIL PRAKTIK

1. Implementasi

a. Struktur Folder dan Database

Folder app/ berada di luar public/ agar kode sumber tidak dapat diakses langsung dari browser. Database db_workshop memiliki tabel mahasiswa (id, nim, nama, prodi, created_at) dan tabel buku untuk tugas mandiri.



b. Fungsi Setiap File

File	Fungsi
Database.php	Membuka koneksi PDO ke db_workshop satu kali (Singleton)
Mahasiswa.php (Model)	Query tabel mahasiswa: getAll, find, create, update, delete (prepared statement)
MahasiswaController.php	index, store, update, destroy: memeriksa input, memanggil Model, mengirim JSON
autoload.php	Memuat class otomatis dari folder config, core, models, controllers
Router.php	Membaca URL dan method HTTP, lalu memanggil Controller yang sesuai
index.php	Front Controller: memuat autoloader lalu menjalankan Router
.htaccess	Meneruskan semua request ke index.php sehingga URL tanpa .php

Tabel 1. Fungsi setiap file

c. Kode Utama

Router memetakan nama resource pada URL ke Controller, lalu memilih method Controller berdasarkan HTTP method.

```php
<?php
class Router {
    // Peta sederhana: nama resource pada URL => nama class Controller
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController', // Tugas Praktik Mandiri 1
    ];

    public function dispatch(): void {
        $uri = trim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/');
        $segments = explode('/', $uri);
        $resource = end($segments) ?: 'mahasiswa';
        $controllerName = $this->map[$resource] ?? null;

        if (!$controllerName) {
            http_response_code(404);
            header('Content-Type: application/json');
            echo json_encode(['success' => false, 'message' => 'Endpoint
tidak ditemukan']);
            exit;
        }

        $controller = new $controllerName();
        switch ($_SERVER['REQUEST_METHOD']) {
            case 'GET': $controller->index(); break;
            case 'POST': $controller->store(); break;
            case 'PUT': $controller->update($_GET['id'] ?? null); break;
            case 'DELETE': $controller->destroy($_GET['id'] ?? null);
break;
            default:
                http_response_code(405);
                header('Content-Type: application/json');
                echo json_encode(['success' => false, 'message' => 'Method
tidak diizinkan']);
        }
    }
}
```

Contoh Model: method create() memakai prepared statement dan validasi NIM duplikat (tugas mandiri 2).

```
    public function create(array $data): string {
        if ($this->existsByNim($data['nim'])) {
            throw new DomainException('NIM sudah terdaftar');
        }
        $stmt = $this->db->prepare(
            'INSERT INTO mahasiswa (nim, nama, prodi) VALUES (:nim, :nama,
:prodi)'
        );
        $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'],
':prodi' =>
            $data['prodi']]);
        return $this->db->lastInsertId();
    }
```

Contoh Controller: store() memeriksa field wajib (400), memanggil Model, dan membalas 201, atau 409 bila NIM sudah terdaftar.

```
    public function store(): void {
        header('Content-Type: application/json');
        $data = json_decode(file_get_contents('php://input'), true);
        if (empty($data['nim']) || empty($data['nama']) ||
empty($data['prodi'])) {
            http_response_code(400);
            echo json_encode(['success' => false, 'message' => 'Field nim,
nama, dan prodi wajib diisi']);
            exit;
        }
        try {
            $id = $this->model->create($data);
        } catch (DomainException $e) {
            // Validasi Model (Tugas Praktik Mandiri 2): NIM duplikat
            http_response_code(409);
            echo json_encode(['success' => false, 'message' => $e-
>getMessage()]);
            exit;
        }
        http_response_code(201);
        echo json_encode(['success' => true, 'message' => 'Data berhasil
ditambahkan',
            'data' => ['id' => $id]]);
    }
```

2. Tugas Praktik Mandiri

a. Resource Buku

Dibuat tabel buku, Buku.php (Model), dan BukuController.php dengan pola yang sama, lalu didaftarkan pada peta Router: 'buku' => 'BukuController'.

b. Validasi NIM Duplikat

Model Mahasiswa memiliki method existsByNim(); create() dan update() melempar DomainException bila NIM sudah ada, dan Controller membalas 409 Conflict.

Alamat diawali http://localhost/setoran_BKPM-1/si-akademik_acara17/public. Body dan response berformat JSON.

Method	Alamat	Contoh request	Response / status
POST	/mahasiswa	{"nim":"2201001","nama":"Andi Saputra","prodi":"Teknik Informatika"}	201 Created; 400 bila field kosong; 409 bila NIM sudah ada
GET	/mahasiswa	-	200: seluruh data
GET	/mahasiswa?id=1	-	200: satu data; 404 bila tidak ada
PUT	/mahasiswa?id=1	{"nim":"2201001","nama":"Andi Saputra Wijaya","prodi":"Teknik Informatika"}	200; 409 bila NIM milik data lain
DELETE	/mahasiswa?id=1	-	200: data dihapus
POST/GET/PUT/DELETE	/buku	{"judul":"Pemrograman PHP Dasar","penulis":"Rina Putri","tahun":2024}	sama seperti mahasiswa
Lainnya	resource tidak terdaftar / method lain	-	404 Endpoint tidak ditemukan; 405 Method tidak diizinkan

Tabel 2. Dokumentasi endpoint

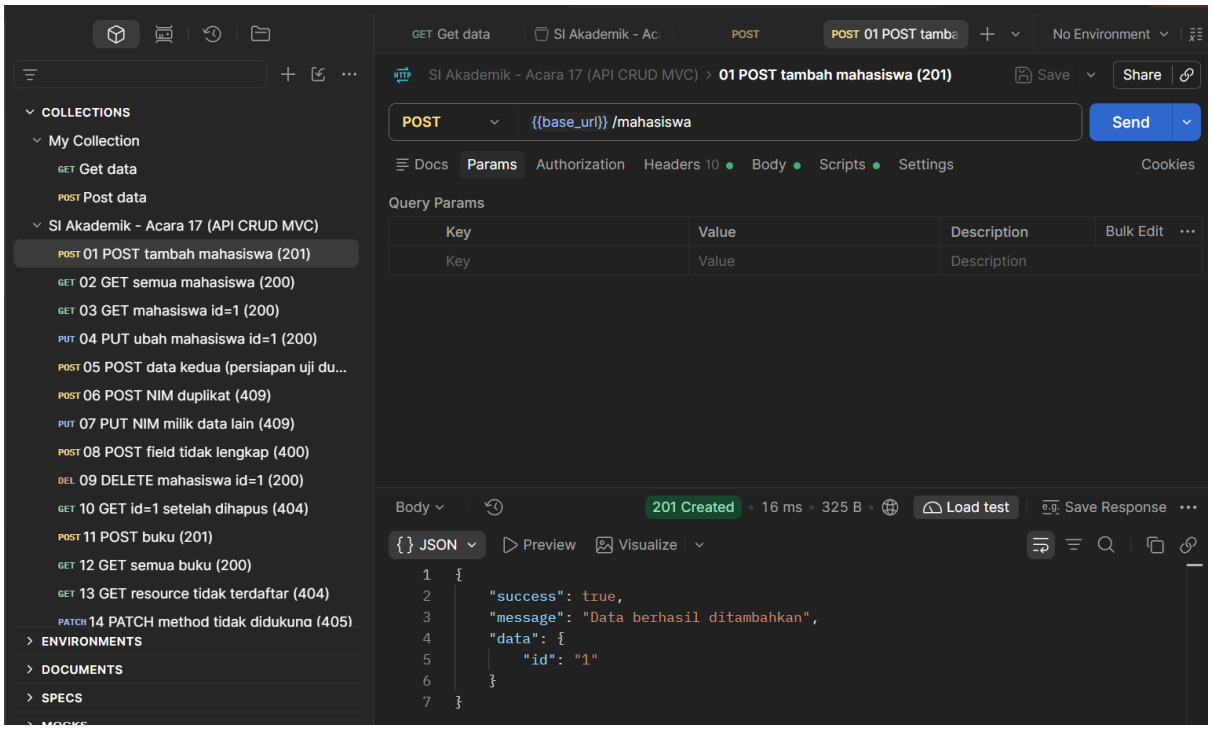
3. Pengujian API

Pengujian mengikuti Langkah 9 modul: POST, GET seluruh data, GET ?id=1, PUT ?id=1, DELETE ?id=1, lalu GET kembali (404). Ditambah uji NIM duplikat dan resource Buku. Seluruh skenario menghasilkan status yang diharapkan.

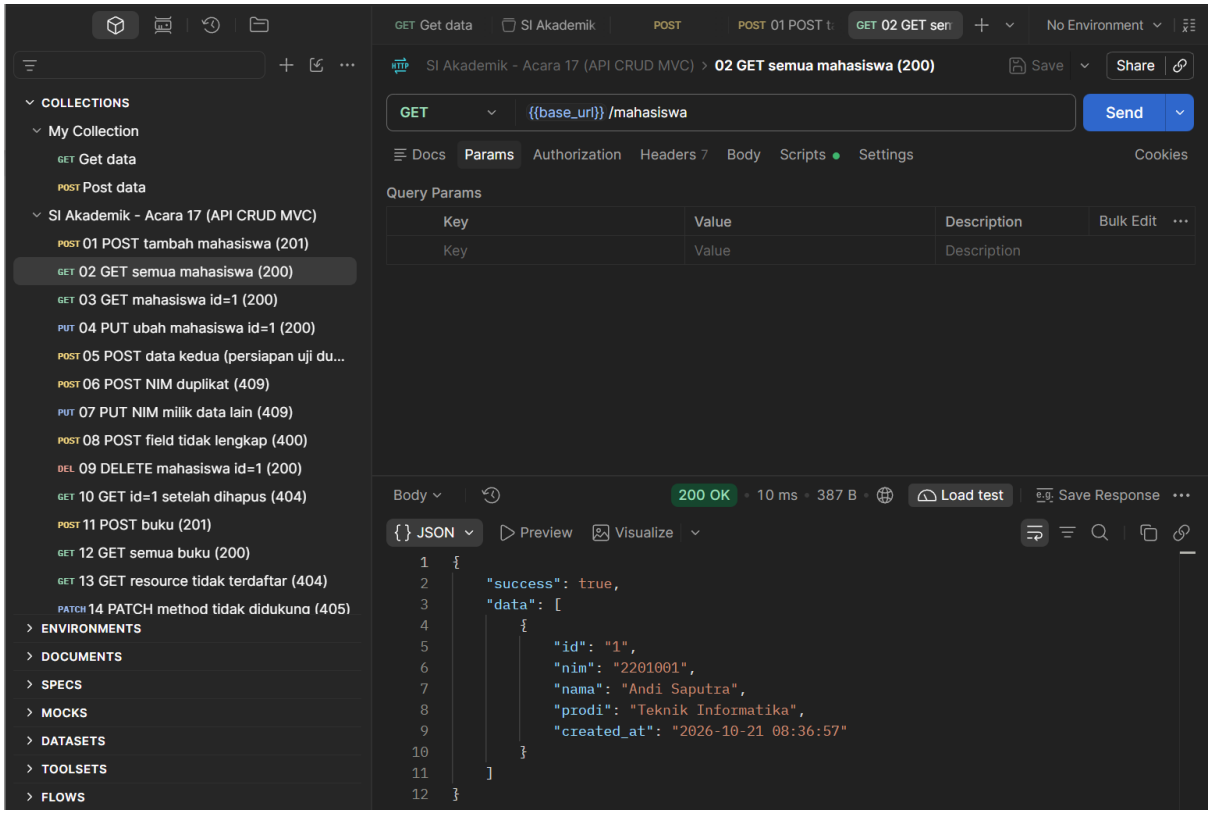
No	Skenario	Request	Status	Gambar
1	POST data valid	POST /mahasiswa	201 Created	2
2	GET seluruh data	GET /mahasiswa	200 OK	3

No	Skenario	Request	Status	Gambar
3	GET satu data	GET /mahasiswa?id=1	200 OK	4
4	PUT ubah data	PUT /mahasiswa?id=1	200 OK	5
5	DELETE hapus data	DELETE /mahasiswa?id=1	200 OK	6
6	GET setelah dihapus	GET /mahasiswa?id=1	404 Not Found	7
7	POST NIM duplikat	POST /mahasiswa	409 Conflict	8
8	POST buku	POST /buku	201 Created	9

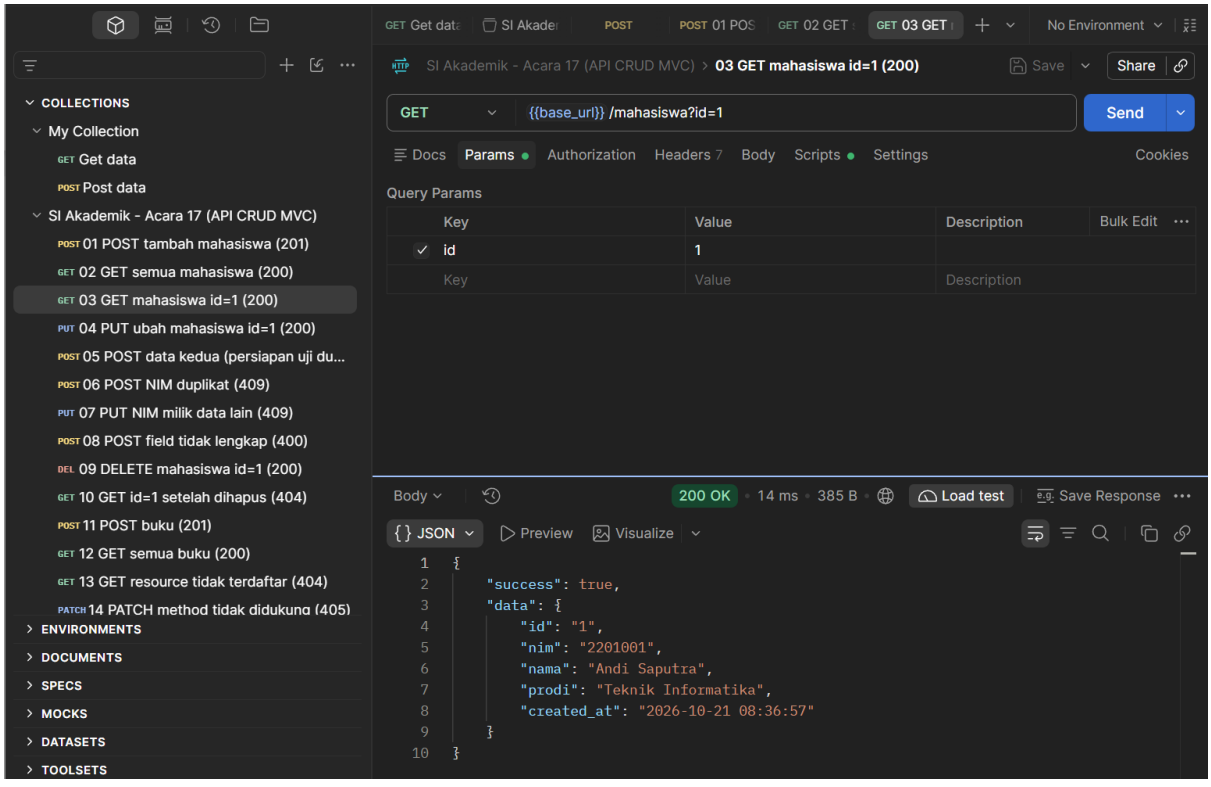
Tabel 3. Hasil pengujian



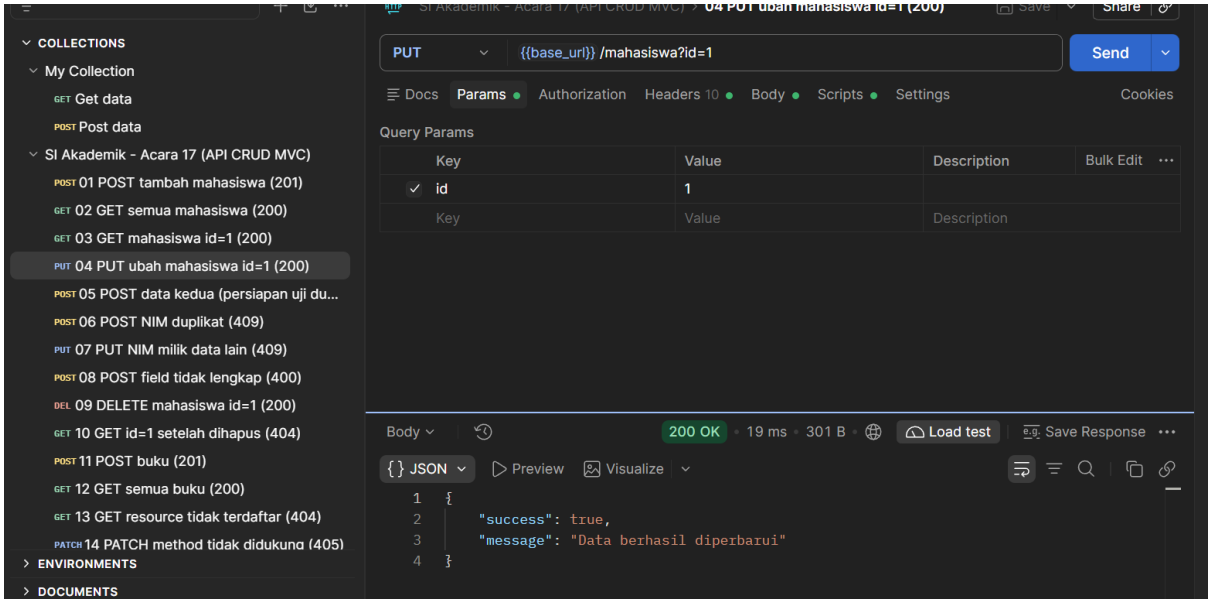
Gambar 2. POST /mahasiswa dengan data valid: status 201 Created



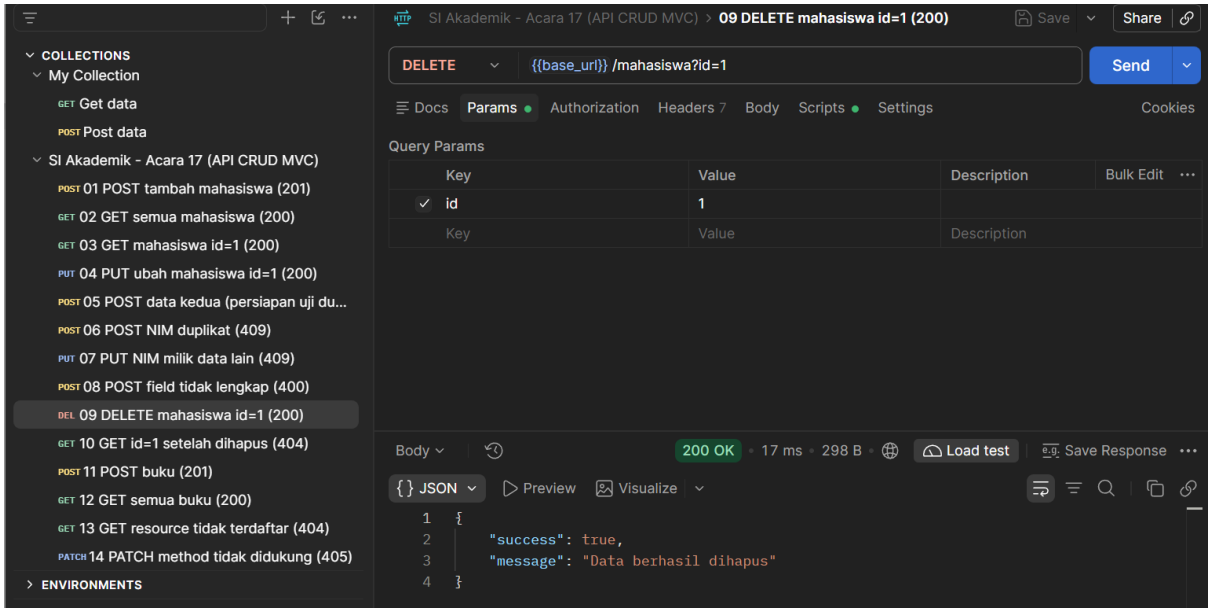
Gambar 3. GET /mahasiswa: seluruh data, status 200 OK



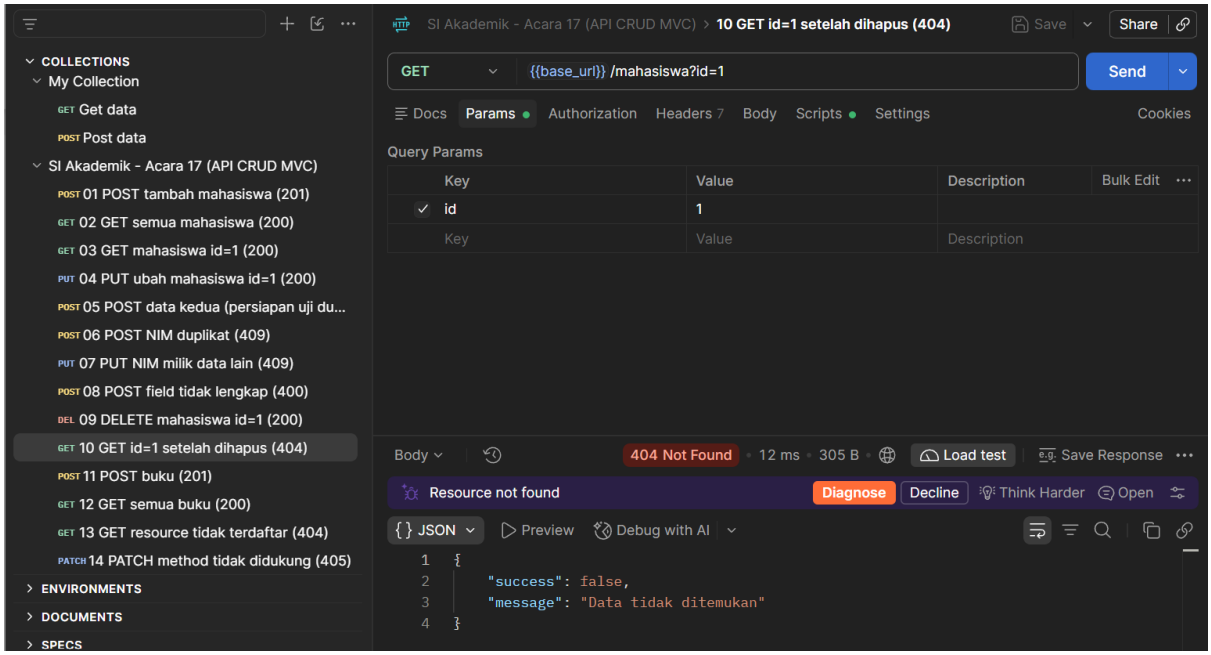
Gambar 4. GET /mahasiswa?id=1: satu data, status 200 OK



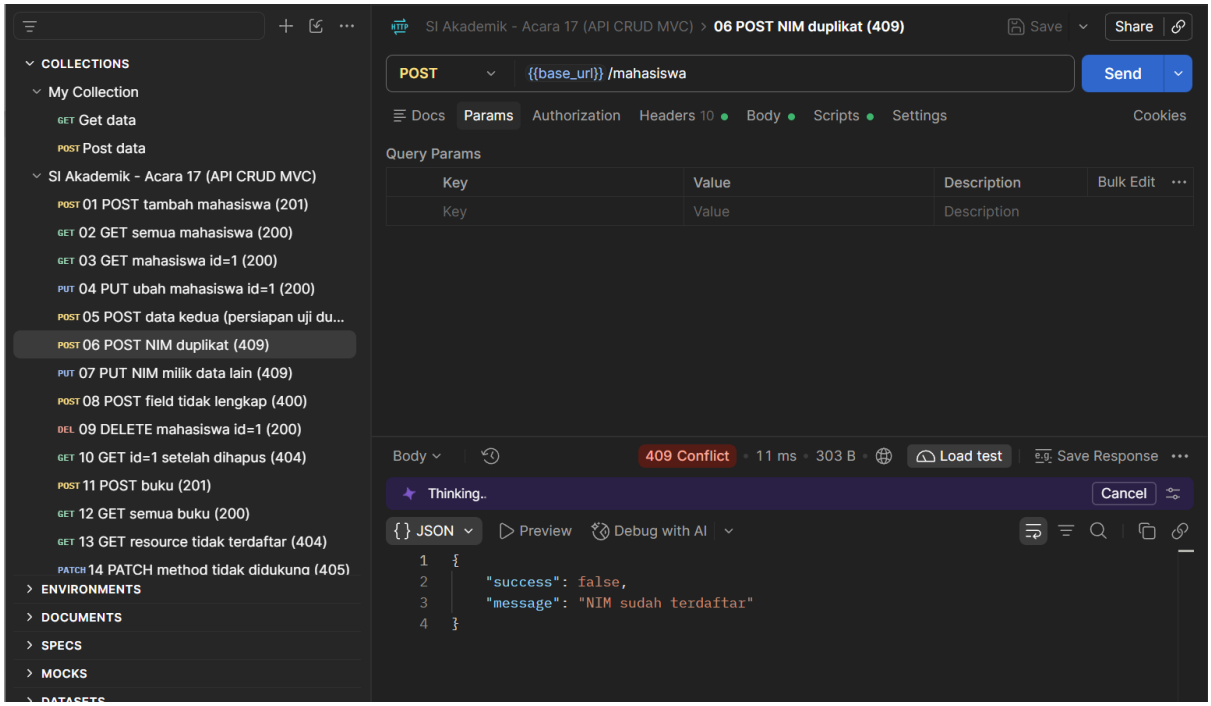
Gambar 5. PUT /mahasiswa?id=1: data diperbarui, status 200 OK



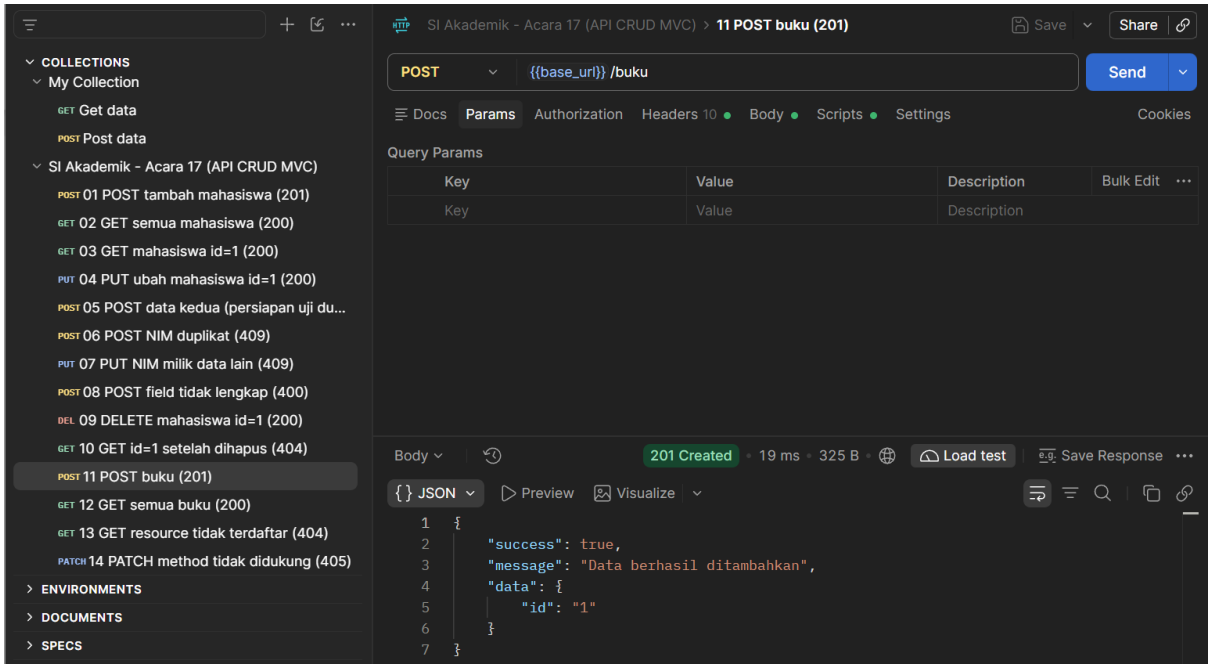
Gambar 6. DELETE /mahasiswa?id=1: data dihapus, status 200 OK



Gambar 7. GET /mahasiswa?id=1 setelah dihapus: status 404 Not Found



Gambar 8. POST dengan NIM yang sudah terdaftar: status 409 Conflict



Gambar 9. POST /buku: resource Buku, status 201 Created

4. Pembahasan

Pemisahan MVC membuat tugas tiap bagian jelas: Router mengarahkan, Controller mengatur alur dan response, Model satu-satunya tempat query. Karena itu resource Buku cukup menambah Model, Controller, dan satu baris Router. Prepared statement mencegah SQL Injection, dan folder app/ di luar public/ melindungi kode sumber. Kekurangan yang dibiarkan sesuai modul: PUT dan DELETE untuk id yang tidak ada tetap membalas 200, dan format response masih ditulis di tiap Controller (dirapikan pada acara 18).

KESIMPULAN

1. Kerangka MVC PHP Native berhasil dibangun: Model, Controller, Autoloader, Router, dan Front Controller.

2. API CRUD mahasiswa berhasil diuji: POST 201, GET 200, PUT 200, DELETE 200, dan GET setelah dihapus 404.

3. Tugas mandiri selesai: resource Buku, validasi NIM duplikat (409), dokumentasi endpoint, dan diagram alur.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.